

Parallel 3.1 Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

0xStalin

100proof

T1MOH

Alix40

Assisting Auditors

Alexzoid (Formal Verification)

March 4, 2026

Contents

1 About Cyfrin **2**

2 About This Document **2**

3 Disclaimer **2**

4 Copyright and License **2**

5 Audit Overview **3**

6 Audit Scope **3**

7 Risk Classification **4**

8 Protocol Summary **4**

8.1 Key Modules & Architecture 4

8.2 Architecture Summary 5

9 Attestation **5**

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Parallel on Parallel 3.1. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated March 4, 2026, was delivered to Parallel, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Parallel or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 15 days, the Cyfrin team conducted an audit on the Parallel 3.1 code provided by Parallel. The findings consist of 1 High, 5 Medium & 10 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Parallel 3.1
Repository	parallel-core
Commit	b439bedb0ffb...
Repository 2	parrallel-tokens
Commit	b16007cec636...
Fix Commit	c832d8295357...
Repository 3	parallel-parallelizer
Commit	20674ad9a3a8...
Fix Commit	6735f32b21b8...
Audit Timeline	Feb 9th - Feb 27th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	5	3	2
Low Risk	10	6	4
Informational	10	5	5
Gas Optimizations	1	0	1
Total	27	15	12

6 Audit Scope

Parallel v3.0 is already deployed to production; the scope of this audit was Parallel version 3.1, an upgrade to allow distributing yield generated by yield-bearing collaterals in the system backing the minted stablecoin.

All contracts in the following repositories were in scope, at these commits:

- parallel-core: b439bedb0ffb8b1780fa666f78e23dbdf156f1ae
- parrallel-tokens: b16007cec636f563c8a21e78349893b6a3720522
- parallel-parallelizer: 20674ad9a3a82bdeb50f9bf2b8248dad13c872b1

The in-scope folders:

```
parallel-core/contracts/*
parrallel-tokens/contracts/*
parallel-parallelizer/contracts/*
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

Parallel is a decentralized, modular, over-collateralized stablecoin protocol designed for the EVM ecosystem. It enables issuance of fully decentralized stablecoins, starting with `USDp`, backed by a diversified, yield-generating basket of correlated assets. The protocol prioritizes capital efficiency, sustainable native yield, and strong peg stability while operating natively across multiple chains.

8.1 Key Modules & Architecture

1. Parallelizer (Core Engine)
   - Diamond-proxy implementation (forked from Angle Protocol's Transmuter)
   - Handles minting, burning, redemption, and collateral management as well as surplus distribution for yield-bearing collaterals
   - Supports dynamic piecewise-linear fee curves for mint/burn operations
   - Uses Chainlink, Morpho, or custom oracles for price feeds
   - Maintains accounting via a global `normalizer` mechanism
   - Features surplus harvesting, collateral rebalancing, and redemption penalty curves
2. Savings Module (Yield Distribution)
   - ERC-4626 compliant vaults (e.g. `sUSDp` for `USDp` holders)
   - Users deposit stablecoins to earn protocol-generated yield
   - Yield sources include:
     - Collateral surplus monetization
     - Mint/burn/redemption fees
     - Bridge module fees
     - Returns on yield-bearing backing assets
   - Yield is distributed through a per-second inflation rate (APY)
3. Bridging Module (Cross-Chain)
   - LayerZero OFT-based `BridgeableTokenP` contracts on non-Ethereum chains
   - Principal-token vs. OFT swapping logic

4. Harvesters & Rebalancing
   - `GenericHarvester` + `MultiBlockHarvester`
   - Rebalance collateral exposures
   - Use flashloans for atomic operations

8.2 Architecture Summary

The diamond proxy delegates calls to facet contracts which read and update a centralized `ParallelizerStorage` via shared library accessors; facets in turn call helper libraries for pricing, manager interactions, and surplus calculations. `tokenP` is the canonical stablecoin contract used by facets for minting and burning, while flashloan and bridge token variants extend `tokenP` functionality for specialized flows. The `normalizer` and `normalizedStables` variables form the protocol's core issuance accounting: normalized units are tracked per collateral and scaled by the normalizer to compute actual outstanding stablecoin supply.

Managers and harvesters move assets between reserves and external yield sources. Oracle modules provide pricing to the mint, burn, redemption and surplus flows. State is kept centralized in the diamond storage, with pluggable integrations for collateral, strategies, savings, flash loans, and bridging.

9 Attestation

Cyfrin attests that it performed a security review of the Parallel 3.1 code provided by Parallel.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-03-04-cyfrin-parallel3.1-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Parallel has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=75a80dd15005a17f4179ec78334c0c2229ef2256428b87011f2f4bc76d8200b0
F=2026-03-04-cyfrin-parallel3.1-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.